

第二章：批处理系统

引言

本章导读

本章展现了操作系统一系列功能：

- 通过批处理支持多个程序的自动加载和运行
- 操作系统利用硬件特权级机制，实现对操作系统自身的保护

上一章，我们在 RV64 裸机平台上成功运行起来了 `Hello, world!` 并成功实现了染色的过程。看起来这个过程非常顺利，只需要一条命令就能全部完成。但实际上，在那个计算机刚刚诞生的年代，很多事情并不像我们想象的那么简单。当时，程序被记录在打孔的卡片上，使用汇编语言甚至机器语言来编写。而稀缺且昂贵的计算机由专业的管理员负责操作，就和我们在上一章所做的事情一样，他们手动将卡片输入计算机，等待程序运行结束或者终止程序的运行。最后，他们从计算机的输出端——也就是打印机中取出程序的输出并交给正在休息室等待的程序提交者。

实际上，这样做是一种对于珍贵的计算资源的浪费。因为当时的计算机和今天的个人计算机不同，它的体积极其庞大，能够占满一整个空调房间，像巨大的史前生物。管理员在房间的各个地方跑来跑去、或是等待打印机的输出的这些时间段，计算机都并没有在工作。于是，人们希望计算机能够不间断的工作且专注于计算任务本身。

批处理系统 (Batch System) 应运而生。它的核心思想是：将多个程序打包到一起输入计算机。而当一个程序运行结束后，计算机会**自动**加载下一个程序到内存并开始执行。这便是最早真正意义上的操作系统。

程序总是难免出现错误。但人们希望一个程序的错误不要影响到操作系统本身，它只需要终止出错的程序，转而运行执行序列中的下一个程序即可。如果后面的程序都无法运行就太糟糕了。这种**保护**操作系统不受有意或无意出错的程序破坏的机制被称为**特权级 (Privilege)** 机制，它实现了用户态和内核态的隔离，需要软件和硬件的共同努力。

本章我们的主要目的也是设计一个批处理的操作系统。毕竟将待执行的程序嵌入main.c之中是十分粗暴的，也不符合我们对操作系统的认知。这同时也意味着我们将开始使用独立的测例文件，并把它们打包到os之中。

实践体验

本章我们引入了用户程序，为了解耦内核与用户程序，我们分离了两个仓库，分别是存放内核程序的 **uCore-Tutorial-Code-20xxx**（下称代码仓库，最后几位 x 表示学期）与存放用户程序的 **uCore-Tutorial-Test-20xxx**（下称测例仓库）。

首先你需要进入代码仓库文件夹（如果已经执行过该步骤则不需要再重复执行）并 clone 用户程序仓库：

```
$ cd uCore-Tutorial-Code-2025S
$ git clone https://github.com/LearningOS/uCore-Tutorial-Test-2025S.git user
```

上面的指令会将测例仓库克隆到代码仓库下并命名为 `user`，注意 `/user` 在代码仓库的 `.gitignore` 中，因此不会出现 `.git` 文件夹嵌套的问题，并且你 `checkout` 代码仓库时也不会影响测例仓库的内容。

注解：在你切换代码仓库的分支前，请务必先清理代码仓库原分支的编译结果，此时需要执行

```
$ make clean
```

如果你没有清理编译结果便切换了分支，可能出现清理后也无法跑通 base 的情况，此时请尝试切换回原分支并执行清理指令。

如果测例仓库有所更新，你可能需要清理掉测例仓库原版的编译结果，此时需要执行

```
$ make -C user clean
```

它的作用基本等价于如下写法，但是更简便

```
$ cd user
$ make clean
$ cd ..
```

我们可以通过 `make user` 生成用户程序，最终将 `.bin` 文件放在 `user/target/bin` 目录下。

```
$ git checkout ch2
$ make user BASE=1 CHAPTER=2
$ make run
```

也可以直接运行打包好的测试程序。`make test` 会完成 `make user` 和 `make run` 两个步骤（自动设置 `CHAPTER`），我们可以通过 `BASE` 控制是否生成留做练习的测例。

```
$ make test BASE=1
```

如果顺利的话，我们可以看到批处理系统自动加载并运行所有的程序并且正确在程序出错的情况下保护了自身：

```
.-----
|  _  \   | | | |   /      |      | /      ||  _  \ | | | | | | |
|  |_) |   | | | |   |  (----`---|  |----`|  (----`|  |_) || |
|      /   | | | |   \  \   | |      \  \   |  _  < | |
|  |\  \----.|  `--'  |.----)  |      |.----)  |  |_) || |
|  _| `_.----|  \____/  |_____/      |  _|  |_____/  |_____/ | |

[rustsbi] Platform: QEMU (Version 0.1.0)
[rustsbi] misa: RV64ACDFIMSU
[rustsbi] mideleg: 0x222
[rustsbi] medeleg: 0xb1ab
[rustsbi-dtb] Hart count: cluster0 with 1 cores
[rustsbi] kernel entry: 0x80200000
hello wrold!
Hello world from user mode program!
Test hello_world OK!
3^10000=5079
...
ALL DONE
```

可以看到 4 个基础测试程序都可以正常运行。

本章代码导读

相比于上一章的操作系统，本章操作系统有两个最大的不同之处，一个是支持应用程序在用户态运行，且能完成应用程序发出的系统调用；另一个是能够一个接一个地自动运行不同的应用程序。

首先改进应用程序，让它能够在用户态执行，并能发出系统调用。这其实就是本章中 **实现应用程序以及user文件夹** 小节介绍内容。具体而言，编写多个应用程序，修改编译应用所需的 `linker.ld` 文件来**调整程序的内存布局**，让操作系统能够把应用加载到指定内存地址后顺利启动并运行应用程序。

应用程序运行中，操作系统要支持应用程序的输出功能，并还能支持应用程序退出。这需要完成 `sys_write` 和 `sys_exit` 系统调用访问请求的实现。

特权级机制

本节导读

为了保护我们的批处理操作系统不受到出错应用程序的影响并全程稳定工作，单凭软件实现是很难做到的，而是需要CPU提供一种特权级隔离机制，使CPU在执行应用程序和操作系统内核的指令时处于不同的特权级。本节主要介绍了特权级机制的软硬件设计思路，以及RISC-V的特权级架构，包括特权指令的描述。

特权级的软硬件协同设计

实现特权级机制的根本原因是应用程序运行的安全性不可充分信任。在上一节里，操作系统和应用紧密连接在一起，形成一个应用程序来执行。随着应用需求的增加，操作系统也越来越大，会以库的形式存在；同时应用自身也会越来越复杂。由于操作系统会给多个应用提供服务，所以它可能的错误会比较快地被发现，但应用自身的错误可能就不会很快发现。由于二者通过编译器形成一个应用程序来执行，即使是应用本身的问题，也会导致操作系统受到连累，从而可能导致整个计算机系统都不可用了。

所以，计算机专家就想到一个方法，能否让相对安全可靠的操作系统不受到应用程序的破坏，运行在一个安全的执行环境中，而让应用程序运行在一个无法破坏操作系统的执行环境中？

为确保操作系统的安全，对应用程序而言，需要限制的主要有两个方面：

- 应用程序不能访问任意的地址空间（这个在第四章会进一步讲解，本章不会讲解）
- 应用程序不能执行某些可能破坏计算机系统的指令（本章的重点）

假设有了这样的限制，我们还需要确保应用程序能够得到操作系统的服务，即应用程序和操作系统还需要有交互的手段。使得低特权级软件都只能做高特权级软件允许它做的，且低特权级软件的超出其能力的要求必须寻求高特权级软件的帮助。在这里的高特权级软件就是低特权级软件的软件执行环境。

为了完成这样的特权级需求，需要进行软硬件协同设计。一个比较简洁的方法就是，处理器设置两个不同安全等级的执行环境：用户态特权级的执行环境和内核态特权级的执行环境。且明确指出可能破坏计算机系统的内核态特权级指令子集，规定内核态特权级指令子集中的指令只能在内核态特权级的执行环境中执行，如果在用户态特权级的执行环境中执行这些指令，会产生异常。处理器在执行不同特权级的执行环境下的指令前进行特权级安全检查。

为了让应用程序获得操作系统的函数服务，采用传统的函数调用方式（即通常的 `call` 和 `ret` 指令或指令组合）将会直接绕过硬件的特权级保护检查。所以要设计新的指令：**执行环境调用**（Execution Environment Call，简称 `ecall`）和**执行环境返回**（Execution Environment Return，简称 `eret`）：

- **ecall**：具有用户态到内核态的执行环境切换能力的函数调用指令（RISC-V中就有这条指令）
- **eret**：具有内核态到用户态的执行环境切换能力的函数返回指令（RISC-V中有类似的 `sret` 指令）

但硬件具有了这样的机制后，还需要操作系统的配合才能最终完成对操作系统自己的保护。首先，操作系统需要提供相应的控制流，能在执行 `eret` 前准备和恢复用户态执行应用程序的上下文。其次，在应用程序调用 `ecall` 指令后，能够保存用户态执行应用程序的上下文，便于后续的恢复；且还要坚持应用程序发出的服务请求是安全的。

注解：在实际的CPU，如x86、RISC-V等，设计了多达4种特权级。对于一般的操作系统而言，其实只要两种特权级就够了。

RISC-V 特权级架构

RISC-V 架构中一共定义了 4 种特权级：

级别	编码	名称
0	00	用户/应用模式 (U, User/Application)
1	01	监督模式 (S, Supervisor)
2	10	H, Hypervisor
3	11	机器模式 (M, Machine)

其中，级别的数值越大，特权级越高，掌控硬件的能力越强。从表中可以看出，M 模式处在最高的特权级，而 U 模式处于最低的特权级。

之前我们给出过支持应用程序运行的一套**执行环境栈**，现在我们站在特权级架构的角度去重新看待它：

- 内核代码运行在 S 模式上
- 应用程序运行在 U 模式上
- 运行在 M 模式上的软件被称为**监督模式执行环境** (SEE, Supervisor Execution Environment)

注解：按需实现 RISC-V 特权级

RISC-V 架构中，只有 M 模式是必须实现的，剩下的特权级则可以根据跑在 CPU 上应用的实际需求进行调整：

- 简单的嵌入式应用只需要实现 M 模式
- 带有一定保护能力的嵌入式系统需要实现 M/U 模式
- 复杂的多任务系统则需要实现 M/S/U 模式

到目前为止，(Hypervisor, H)模式的特权规范还没完全制定好。所以本书不会涉及。

回顾第一章，当时只是实现了简单的支持单个裸机应用的库级别的"三叶虫"操作系统，它和应用程序全程运行在 S 模式下，应用程序很容易破坏没有任何保护的执行环境-操作系统。而在后续的章节中，我们会涉及到RISC-V的 M/S/U 三种特权级：

- 应用程序和用户态支持库运行在 U 模式的最低特权级
- 操作系统内核运行在 S 模式特权级（在本章表现为一个简单的批处理系统），形成支撑应用程序和用户态支持库的执行环境
- RustSBI 实际上是运行在更底层的 M 模式特权级下的软件，是操作系统内核的执行环境

整个软件系统就由这三层运行在不同特权级下的不同软件组成。

异常控制流

执行环境的另一种功能是对上层软件的执行进行监控管理。监控管理可以理解为，当上层软件执行的时候出现了一些情况导致需要用到执行环境中提供的功能，因此需要暂停上层软件的执行，转而运行执行环境的代码。由于上层软件和执行环境被设计为运行在不同的特权级，这个过程也往往（而**不一定**）伴随着 CPU 的**特权级切换**。当执行环境的代码运行结束后，我们需要回到上层软件暂停的位置继续执行。在 RISC-V 架构中，这种与常规控制流（顺序、循环、分支、函数调用）不同的**异常控制流** (ECF, Exception Control Flow) 被称为**异常（Exception）**。

用户态应用直接触发从用户态到内核态的**异常控制流**的原因总体上可以分为两种：执行**Trap类异常**指令和执行了会产生**Fault类异常**的指令。

Trap类异常指令 就是指用户态软件为获得内核态操作系统的服务功能而发出的特殊指令。

Fault类的指令是指用户态软件执行了在内核态操作系统看来是非法操作的指令。

下表中我们给出了 RISC-V 特权级定义的会导致从低特权级到高特权级的各种**异常**：

Interrupt	Exception Code	Description
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	15	Store/AMO page fault

其中断点(Breakpoint)和**执行环境调用** (Environment call) 两个异常（为了与其他非有意为之的异常区分，会把这种有意为之的指令称为**陷入**或 **trap** 类指令）是通过在上层软件中执行一条特定的指令触发的：当执行 `ebreak` 这条指令的之后就会触发断点陷入异常；而执行 `ecall` 这条指令的时候则会随着 CPU 当前所处特权级而触发不同的**陷入**情况。从表中可以看出，当 CPU 分别处于 M/S/U 三种特权级时执行 `ecall` 这条指令会触发三种陷入。

执行环境调用 `ecall` 是一种很特殊的会产生**陷入**的指令。相邻两特权级软件之间的接口正是基于这种陷入机制实现的：

- M 模式软件 SEE 和 S 模式的内核之间的接口被称为**监督模式二进制接口** (Supervisor Binary Interface, SBI)

- 内核和 U 模式的应用程序之间的接口被称为**应用程序二进制接口** (Application Binary Interface, ABI)，当然它有一个更加通俗的名字——**系统调用** (syscall, System Call)

而之所以叫做二进制接口，是因为它和在同一种编程语言内部调用接口不同，是汇编指令级的一种接口。

RISC-V的特权指令

与特权级无关的一般的指令和通用寄存器 `x0~x31` 在任何特权级都可以任意执行。而每个特权级都对应一些特殊指令和**控制状态寄存器** (CSR, Control and Status Register)，来控制该特权级的某些行为并描述其状态。

如果低优先级下的处理器执行了高优先级的指令，会产生非法指令错误的异常，于是位于高特权级的执行环境能够得知低优先级的软件出现了该错误，这个错误一般是不可恢复的，此时一般它会将上层的低特权级软件终止。

在RISC-V中，会有两类低优先级U模式下运行高优先级S模式的指令：

1. 指令本身属于高特权级的指令，如 `sret` 指令（表示从S模式返回到U模式）
2. 指令访问了 **S模式特权级下才能访问的寄存器** 或内存，如表示S模式系统状态的**控制状态寄存器** `sstatus` 等

RISC-V S模式特权指令：

指令	含义
<code>sret</code>	从S模式返回U模式。在U模式下执行会产生非法指令异常
<code>wfi</code>	处理器在空闲时进入低功耗状态等待中断。在U模式下执行会尝试非法指令异常
<code>sfence.vma</code>	刷新TLB缓存。在U模式下执行会尝试非法指令异常
访问S模式CSR的指令	通过访问 <code>sepc/stvec/scause/sscartch/stval/sstatus/satp</code> 等CSR 来改变系统状态。在U模式下执行会尝试非法指令异常

实现应用程序以及user文件夹

本节导读

本节主要讲解如何设计实现被批处理系统逐个加载并运行的应用程序。它们是假定在 U 特权级模式运行的前提下而设计、编写的。实际上，如果应用程序的代码都符合它要运行的某特权级的约束，那它完全可能在某特权级中运行。保证应用程序的代码在 U 模式运行是我们接下来将实现的批处理系统的任务。其涉及的设计实现要点是：

- 应用程序的内存布局
- 应用程序发出的系统调用

从某种程度上讲，这里设计的应用程序与第一章中的最小用户态执行环境有很多相同的地方。即设计一个应用程序，能够在用户态通过操作系统提供的服务完成自身的功能。

user文件夹以及测例简介

本章我们引入了用户程序。为了将内核与应用解耦，我们将二者分成了两个仓库，分别是存放内核程序的 **uCore-Tutorial-Code-20xxx**（下称代码仓库，最后几位 x 表示学期）与存放用户程序的 **uCore-Tutorial-Test-20xxx**（下称测例仓库）。

你首先需要进入代码仓库文件夹并 clone 用户程序仓库（如果已经执行过该步骤则不需要再重复执行）：


```
$ git clone https://github.com/LearningOS/uCore-Tutorial-Code-2025S.git
$ cd uCore-Tutorial-Code-2025S
$ git checkout ch2
$ git clone https://github.com/LearningOS/uCore-Tutorial-Test-2025S.git user
```

上面的指令会将测例仓库克隆到代码仓库下并命名为 `user`，注意 `/user` 在代码仓库的 `.gitignore` 文件中，因此不会出现 `.git` 文件夹嵌套的问题，并且你在代码仓库进行 `checkout` 操作时也不会影响测例仓库的内容。

测例实际就是批处理操作系统中一个个待执行的文件。下面我们看一个测例来理解本章以及之后测例的本质：

```
// ch2_hello_world.c

#include <stdio.h>
#include <unistd.h>

int main(void)
{
    puts("Hello world from user mode program!\nTest hello_world OK!");
    return 0;
}
```

这个测例编译出来实际上就是一个可执行的打印helloworld的程序。如果是windows或者linux上它编译之后是可以直接执行的。它也可以用来检查我们操作系统的实现是否有问题。

我们的测例是通过cmake来编译的。具体编译出测例的指令可以参见其中的readme。在使用测例的时候要注意，由于我们使用的是自己的os系统，因此所有常见的C库，比如stdlib.h，stdio.h等等都不能使用C官方的版本。这里在user的include和lib之中我们提供了搭配本次实验的对应库，里面实现了所有测例所需要的函数。

user的库是如何调用到os的系统调用的呢？在user/lib/arch/riscv下的syscall_arch.h为我们包装好了使用riscv汇编调用系统调用ecall的函数接口。lib之中的syscall.c文件就是用这些包装好的函数来进行系统调用实现完整的函数功能。在第一章中大家已经了解了异常委托的机制。U态的ecall指令会转到S态，也就是我们编写的os来进行处理，这样整个逻辑就打通了：为了使得测例成功运行，我们必须实现处理对应ecall的函数。

应用程序的ecall处理流程

ecall作为异常的一种，操作系统和CPU对它的处理方式其实和其他各种异常没什么区别。U态进行ecall调用具体的异常编号是8-Environment call from U-mode。

RISCV处理异常需要引入几个特殊的寄存器——CSR寄存器。这些寄存器会记录异常和中断处理流程所需要或保存的各种信息。

几个比较关键的CSR寄存器如下：

- **scause**: 它用于记录异常和中断的原因。它的最高位为1是中断，否则是异常。其低位决定具体的种类。
- **sepc**: 处理完毕中断异常之后需要返回的PC值。
- **stval**: 产生异常的指令的地址。
- **stvec**: 处理异常的函数的起始地址。
- **sstatus**: 记录一些比较重要的状态，比如是否允许中断异常嵌套。

所以当U态执行ecall指令的时候就产生了异常。此时CPU会处理上述的各个CSR寄存器，之后跳转至stvec所指向的地址，也就是我们的异常处理函数。我们的os的这个函数的具体位置是在trap_init函数之中就指定了——是uservec函数。这个函数位于trampoline.S之中，是由汇编语言编写的。在uservec之中，os保存了U态执行流的各个寄存器的值。这些值的位置其实已经由trap.h中的trapframe结构体规定好了：

```
// os/trap.h

struct trapframe {
    /* 0 */ uint64 kernel_satp;    // kernel page table
    /* 8 */ uint64 kernel_sp;      // top of process's kernel stack
    /* 16 */ uint64 kernel_trap;   // usertrap entry
    /* 24 */ uint64 epc;           // saved user program counter
    /* 32 */ uint64 kernel_hartid; // saved kernel tp, unused in our project
    /* 40 */ uint64 ra;
    /* 48 */ uint64 sp;
    /* ... */ ....
    /* 272 */ uint64 t5;
    /* 280 */ uint64 t6;
};
```

由于涉及到直接操作寄存器，因此这里只能使用汇编语言来编写。具体可以参考trampoline.S之中的代码：

```
.section .text
.globl trampoline
trampoline:
.align 4
.globl uservec
uservec:
    #
    # trap.c sets stvec to point here, so
    # traps from user space start here,
    # in supervisor mode, but with a
    # user page table.
    #
    # sscratch points to where the process's p->trapframe is
    # mapped into user space, at TRAPFRAME.
    #

    # swap a0 and sscratch
    # so that a0 is TRAPFRAME
    csrrw a0, sscratch, a0

    # save the user registers in TRAPFRAME
    sd ra, 40(a0)
    ...
    sd t6, 280(a0)

    # save the user a0 in p->trapframe->a0
    csrr t0, sscratch
    sd t0, 112(a0)

    csrr t1, sepc
```



```

sd t1, 24(a0)

ld sp, 8(a0)
ld tp, 32(a0)
ld t1, 0(a0)

# csrw satp, t1
# sfence.vma zero, zero

ld t0, 16(a0)
jr t0

```

然后我们使用jr t0，就跳转到了我们早先设定在 trapframe->kernel_trap 中的地址，也就是 trap.c 之中的 usertrap 函数。这个函数在main的初始化之中已经调用了。

```

// os/trap.c

// set up to take exceptions and traps while in the kernel.
void trapinit(void)
{
    w_stvec((uint64)uservec & ~0x3);    // 写 stvec，最后两位表明跳转模式，该实验始终为 0
}

```

该函数完成异常中断处理与返回，包括执行我们写好的syscall。

从S态返回U态是由 usertrapret 函数实现的。这里设置了返回地址sepc，并调用另外一个 userret 汇编函数来恢复 trapframe 结构体之中的保存的U态执行流数据。

```

void usertrapret(struct trapframe *trapframe, uint64 kstack)
{
    trapframe->kernel_satp = r_satp(); // kernel page table
    trapframe->kernel_sp = kstack + PGSIZE; // process's kernel stack
    trapframe->kernel_trap = (uint64)usertrap;
    trapframe->kernel_hartid = r_tp(); // hartid for cpuid()

    w_sepc(trapframe->epc); // 设置了sepc寄存器的值。

    // set up the registers that trampoline.S's sret will use
    // to get to user space.
    // set S Previous Privilege mode to User.
    uint64 x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    // tell trampoline.S the user page table to switch to.
    // uint64 satp = MAKE_SATP(p->pagetable);

    userret((uint64)trapframe);
}

```

同样由于涉及寄存器的恢复，以及未来页表satp寄存器的设置等，userret也必须是一个汇编函数。它基本上就是uservec函数的镜像，将保存在trapframe之中的数据依次读出用于恢复对应的寄存器，实现恢复用户中断前的状态。

```
.globl userret
userret:
    # userret(TRAPFRAME, pagetable)
    # switch from kernel to user.
    # usertrapret() calls here.
    # a0: TRAPFRAME, in user page table.
    # a1: user page table, for satp.

    # switch to the user page table.在第四章才会有具体作用。
    csrw satp, a1
    sfence.vma zero, zero

    # put the saved user a0 in sscratch, so we
    # can swap it with our a0 (TRAPFRAME) in the last step.
    ld t0, 112(a0)
    csrw sscratch, t0

    # restore all but a0 from TRAPFRAME
    ld ra, 40(a0)
    ld sp, 48(a0)
    ld gp, 56(a0)
    ld tp, 64(a0)
    ld t0, 72(a0)
    ld t1, 80(a0)
    ld t2, 88(a0)
    ...
    ld t4, 264(a0)
    ld t5, 272(a0)
    ld t6, 280(a0)

    # restore user a0, and save TRAPFRAME in sscratch
    csrrw a0, sscratch, a0

    # return to user mode and user pc.
    # usertrapret() set up sstatus and sepc.
    sret
```

需要注意最后执行的sret指令执行了2个事情：从S态回到U态，并将PC移动到sepc指定的位置，继续执行用户程序。

实现批处理操作系统的细节

本节导读

前面一节中我们明白了os是如何执行应用程序的。但是os是如何"找到"这些应用程序并允许它们的呢？在引言之中我们简要介绍了这是由link_app.S以及kernel_app.ld完成的。实际上，能够在批处理操作系统与应用程序之间建立联系的纽带。这主要包括两个方面：

1. **静态编码**：通过一定的编程技巧，把应用程序代码和批处理操作系统代码"绑定"在一起。

2. **动态加载**：基于静态编码留下的"绑定"信息，操作系统可以找到应用程序文件二进制代码的起始地址和长度，并能加载到内存中运行。

这里与硬件相关且比较困难的地方是如何让在内核态的批处理操作系统启动应用程序，且能让应用程序在用户态正常执行。

将应用程序链接到内核

makefile更新

我们首先看一看本章的makefile改变了什么：

```
link_app.o: link_app.S

link_app.S: pack.py
    @$(PY) pack.py

kernel_app.ld: kernelld.py
    @$(PY) kernelld.py

kernel: $(OBSJS) kernel_app.ld link_app.S
    $(LD) $(LDFLAGS) -T kernel_app.ld -o kernel $(OBSJS)
    $(OBJDUMP) -s kernel > kernel.asm
    $(OBJDUMP) -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernel.sym
```

可以看到makefile执行了两个python脚本生成了我们提到的link_app.S和kernel_app.ld。

link_app.S的大致内容如下：

```
.align 4
.section .data
.global _app_num
_app_num:
    .quad 2
    .quad app_0_start
    .quad app_1_start
    .quad app_1_end

.global _app_names
_app_names:
    .string "hello.bin"
    .string "matrix.bin"

.section .data.app0
.global app_0_start
app_0_start:
    .incbin "../user/target/bin/ch2t_write0.bin"

.section .data.app1
.global app_1_start
app_1_start:
    .incbin "../user/target/bin/ch2b_write1.bin"
```

```
app_1_end:
```

pack.py会遍历../user/target/bin，并将该目录下的目标用户程序*.bin包含入 link_app.S中，同时给每一个bin文件记录其地址和名称信息。最后，我们在 Makefile 中会将内核与 link_app.S 一同编译并链接。这样，我们在内核中就可以通过 extern 指令访问到用户程序的所有信息，如其文件名等。

由于 riscv 要求程序指令必须是对齐的，我们对内核链接脚本也作出修改，保证用户程序链接时的指令对齐。这个脚本也会遍历../user/target/，并对每一个bin文件分配对齐的空间。最终修改后的kernel_app.ld脚本中多了如下对齐要求：

```
.data : {
    *(.data) . = ALIGN(0x1000);
    *(.data.app0) . = ALIGN(0x1000);
    *(.data.app1) . = ALIGN(0x1000);
    *(.data.app2) . = ALIGN(0x1000);
    *(.data.app3) . = ALIGN(0x1000);
    *(.data.app4)
    *(.data.*)
}
```

内核的relocation

内核中通过访问 link_app.S 中定义的 _app_num、app_0_start 等符号来获得用户程序位置。

```
// os/loader.c

extern char _app_num[]; // 在link_app.S之中已经定义

void loader_init()
{
    if ((uint64)ekernel >= BASE_ADDRESS) {
        panic("kernel too large...\n");
    }
    app_info_ptr = (uint64 *)_app_num;
    app_cur = -1;
    app_num = *app_info_ptr;
}
```

然而我们并不能直接跳转到 app_n_start 直接运行，因为用户程序在编译的时候，会假定程序处在虚存的特定位置，而由于我们还没有虚存机制，因此我们在运行之前还需要将用户程序加载到规定的物理内存位置。为此我们规定了用户的链接脚本，并在内核完成程序的“搬运”：

```
# user/lib/arch/riscv/user.ld
SECTIONS {
    . = 0x80400000;                # 规定了内存加载位置

    .startup : {
        *crt.S.o(.text)           # 确保程序入口在程序开头
    }

    .text : { *(.text) }
    .data : { *(.data .rodata) }

    /DISCARD/ : { *(.eh_*) }
}
```

这样之后，我们就可以在读取指定内存位置的bin文件来执行它们了。下面是os内核读取link_app.S的info并把它们搬运到0x80400000开始位置的具体过程。

```
// os/loader.c

const uint64 BASE_ADDRESS = 0x80400000, MAX_APP_SIZE = 0x20000;

int load_app(uint64 * info) {
    uint64 start = info[0], end = info[1], length = end - start;
    memset((void *)BASE_ADDRESS, 0, MAX_APP_SIZE);
    memmove((void *)BASE_ADDRESS, (void *)start, length);
    return length;
}
```

用户栈与内核栈

我们自己的OS内核运行时，是需要一个栈来存放自己需要的变量的，这个栈我们称之为内核栈。在RV之中，我们使用sp寄存器来记录当前栈顶的位置。因此，在进入OS之前，我们需要告诉qemu我们OS的内核栈的起始位置。这个在entry.S之中有实现：

```
// entry.S
_entry:
    la sp, boot_stack_top
    call main

.section .bss.stack
.globl boot_stack
boot_stack:
    .space 4096 * 16
    .globl boot_stack_top
```

一个应用程序肯定也需要内存空间来存放执行时需要的种种变量（实际上就是执行程序对应的用户栈），同时我们在上一章节提到了trapframe，这个也需要一个空间存放。那么OS是如何给应用程序分配这些对应的空间的呢？

实际上，我们采用一个静态分配的方式来给程序分配对应的一定大小的空间，并在run_next_app函数初始化应用程序对应的trapframe，并将用户栈对应的起始位置写入trapframe之中的sp寄存器，来让程序找到自己用户栈起始的位置。（注意栈在空间是高到低位，因此这里起始位置的初始化是在静态分配数组的尾部）。

```
// loader.c

__attribute__((aligned(4096))) char user_stack[USER_STACK_SIZE];
__attribute__((aligned(4096))) char trap_page[TRAP_PAGE_SIZE];

int run_next_app()
{
    struct trapframe *trapframe = (struct trapframe *)trap_page;
    ...
    memset(trapframe, 0, 4096);
    trapframe->epc = BASE_ADDRESS;
    trapframe->sp = (uint64)user_stack + USER_STACK_SIZE;
    usertrapret(trapframe, (uint64)boot_stack_top);
    ...
}
```

到这里，一个应用程序就算真正完全加载进入了内存之中进入就绪状态，可以随时运行了。

总结

第二章主要介绍了批处理系统的设计与实现，包括：

1. **特权级机制**：通过RISC-V的特权级架构（M/S/U模式）实现对操作系统的保护
2. **应用程序实现**：设计用户态应用程序，包括内存布局 and 系统调用
3. **批处理系统实现**：通过link_app.S和kernel_app.ld将应用程序与内核绑定，实现自动加载和运行

这些内容为后续章节的多道程序和地址空间管理奠定了基础。